

08 · Symbolic Solver

2026-07-31

Contents

08 · Symbolic Solver	2
1. 设计目标	2
1.1 设计原则	2
2. 组件构成	2
3. Geometry Theorem Database	2
3.1 结构	2
3.2 RAG 检索	3
3.3 规模	3
4. SymPy Engine	3
4.1 用途	3
4.2 接口	3
4.3 示例	3
4.4 数值回填	3
5. Z3 SMT Solver	4
5.1 用途	4
5.2 接口	4
5.3 应用场景	4
6. 自研几何推理引擎	4
6.1 设计动机	4
6.2 规则表示	4
6.3 前向链锁算法	5
6.4 模式匹配	5
6.5 与 LLM 协作	5
7. Lean 形式化校验 (Phase 5 可选)	5
7.1 目标	5
7.2 集成方式	5
7.3 适用阶段	6
8. LLM vs Solver 分工	6
8.1 协作流程	6
9. 失败处理	6
10. 设计路线小结	6

08 · Symbolic Solver

本文档阐述 Symbolic Solver 的组件构成、SymPy/Z3/自研几何引擎/Lean 的集成方式，以及 LLM 与 Solver 的分工。Solver 将 LLM 的规划转化为精确符号计算与证明，保证数学严格性。

1. 设计目标

Solver 的核心目标是**精确性**：所有角度、长度、比例的计算必须符号精确，所有证明步骤必须可验证。LLM 负责”规划”，Solver 负责”证明”。

1.1 设计原则

- **符号优先**：能用符号计算就不用数值近似。
- **可验证**：每步计算可复现，证明可校验。
- **领域专用**：内置几何定理规则，自动应用。
- **失败安全**：无法求解时返回”无法求解”而非编造。

2. 组件构成

组件	角色	技术	详见
Geometry Theorem Database	定理与公式库	结构化存储 + RAG 检索	§ 3
SymPy Engine	代数计算、方程求解、三角	SymPy	§ 4
Z3 SMT Solver	不等式、约束可满足性	Z3	§ 5
自研几何推理引擎	几何专属规则推理（前向链锁）	规则引擎	§ 6
Lean（可选）	形式化证明校验	Lean 4	§ 7

3. Geometry Theorem Database

3.1 结构

每条定理结构化为（前提，结论，适用条件，引用）：

```
{
  "id": "T_inscribed_angle",
  "name": "  圆周角定理",
  "premise": ["On(A,Circle(0))","On(B,Circle(0))","On(C,Circle(0))","On(D,Circle(0))","SameArc(AB,CD)"],
  "conclusion": "Equal(Angle(ACB), Angle(ADB))",
  "condition": "A,B,C,D 共圆且 AB,CD 同弧",
  "category": "circle",
  "reference": "  人教版九年级上册 §24.1"
}
```

3.2 RAG 检索

- 定理库向量化 (embedding, 按 name+premise+conclusion)。
- LLM 根据 DSL 对象/关系检索 Top-K 相关定理。
- 检索结果作为 Prompt [Context] 注入。

3.3 规模

目标 200~500 条, 覆盖: 圆周角/圆心角、切线性质、相似/全等判定、椭圆定义、勾股、正弦/余弦定理等。

4. SymPy Engine

4.1 用途

- 代数方程求解 (如比例 $AB/AC=AE/AD \rightarrow AB*AC=AD*AE$)。
- 三角计算 (角度、正余弦)。
- 表达式化简与变形。
- 多项式运算。

4.2 接口

```
import sympy as sp

def symbolic_solve(equations: list[str], goal: str) -> Solution:
    """ 求解方程组, 验证 goal。 """
    syms = extract_symbols(equations + [goal])
    eqs = [sp.Eq(*parse(eq)) for eq in equations]
    sol = sp.solve(eqs, syms, dict=True)
    goal_expr = parse(goal)
    for s in sol:
        if goal_expr.subs(s) == True or sp.simplify(goal_expr.subs(s)):
            return Solution(verified="true", solution=s)
    return Solution(verified="false", reason=" 无解或不满足目标")
```

4.3 示例

```
# 相似比例 → 目标
symbolic_solve(["AB/AC = AE/AD"], "AB*AC = AD*AE")
# → verified=true, solution={...}
```

4.4 数值回填

对符号解, 用 Graph 中的实际度量数值回填验证 (双重保险), 如 $AB=98.1$, $AC=...$ 代入确认等式成立。

5. Z3 SMT Solver

5.1 用途

- 不等式约束可满足性（如“某角度是否可能为锐角”）。
- 约束求解（如动点范围问题）。
- 反例检查（验证某命题是否恒成立，找反例）。

5.2 接口

```
from z3 import Real, Solver, And, Or

def z3_check_satisfiable(constraints: list[str]) -> str:
    s = Solver()
    vars = {}
    for c in constraints:
        s.add(parse_z3(c, vars))
    result = s.check()
    if result == sat: return "sat"
    elif result == unsat: return "unsat"
    else: return "unknown"
```

5.3 应用场景

- **动点范围**：给定约束，求变量取值范围。
 - **反例搜索**：LLM 提出命题，Z3 快速检查是否存在反例；若 unsat 则命题恒成立。
 - **约束一致性**：检查 Graph 中已知关系是否矛盾。
-

6. 自研几何推理引擎

6.1 设计动机

通用 Solver (SymPy/Z3) 不“懂”几何定理，需 LLM 显式引导。自研引擎内置几何规则，可**前向链锁自动推导**新关系，减轻 LLM 负担。

6.2 规则表示

规则为（前提模式，结论，推导函数）：

规则 R1（同弧圆周角）：

前提： $\text{On}(A, \text{Circle}(O)) \wedge \text{On}(B, \text{Circle}(O)) \wedge \text{On}(C, \text{Circle}(O)) \wedge \text{On}(D, \text{Circle}(O))$
 $\wedge \text{SameArc}(AB, CD)$
结论： $\text{Equal}(\text{Angle}(ACB), \text{Angle}(ADB))$

规则 R2（切线性质）：

前提： $\text{On}(A, \text{Circle}(O)) \wedge \text{Tangent}(\text{Line}(AB), \text{Circle}(O), \text{at}=A)$
结论： $\text{Perpendicular}(OA, AB)$

规则 R3（相似对应边）：

前提: $\text{Similar}(\text{Triangle}(A,B,C), \text{Triangle}(D,E,F))$
结论: $\text{Equal}(\text{Ratio}(AB,DE), \text{Ratio}(BC,EF), \text{Ratio}(AC,DF))$

规则 R4 (勾股):

前提: $\text{Perpendicular}(AB, AC)$
结论: $\text{Equal}(AB^2 + AC^2, BC^2)$

规则 R5 (椭圆定义):

前提: $\text{On}(P, \text{Ellipse}(E)) \wedge \text{Focus}(E, F1, F2)$
结论: $\text{Equal}(\text{Sum}(\text{Dist}(P,F1), \text{Dist}(P,F2)), 2*a)$

6.3 前向链锁算法

```
def forward_chain(graph, rules, max_iter=10):
    changed = True
    iteration = 0
    while changed and iteration < max_iter:
        changed = False
        for rule in rules:
            for binding in match(rule.premise, graph):
                new_rel = rule.conclude(binding)
                if not graph.has(new_rel):
                    graph.add_derived(new_rel, source=rule.id)
                    changed = True
        iteration += 1
    return graph
```

6.4 模式匹配

规则前提含变量 (A,B,C,O), 需在 Graph 上做模式匹配 (图同构子问题)。采用类型过滤 + 索引加速, 避免组合爆炸。

6.5 与 LLM 协作

- LLM 提出规划后, 引擎自动前向链锁, 补全 LLM 可能遗漏的推导。
- 引擎推导的新关系回写 Graph, 供 LLM 复用。
- 引擎无法推导的, LLM 引导 Solver 显式计算。

7. Lean 形式化校验 (Phase 5 可选)

7.1 目标

将证明链翻译为 Lean 4 命题并校验, 达到机器可验证的数学严格性。这是“可验证”原则的最高阶实现。

7.2 集成方式

- 维护 Geometry Graph $\rightarrow$ Lean 命题的翻译器。
- 每条已验证关系翻译为 Lean hypothesis。
- 每步证明翻译为 Lean tactic。

- 调用 Lean 编译器校验，返回通过/失败 + 错误位置。

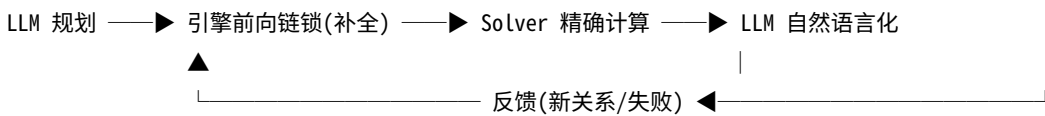
7.3 适用阶段

Phase 5 引入，前期不依赖。Lean 集成成本高但收益（严格性）大，作为系统成熟期的升级。

8. LLM vs Solver 分工

任务	执行者	理由
理解题意、识别题型	LLM	需语义理解
定理选择与规划	LLM	需启发式判断
角度/长度精确计算	Solver (SymPy)	需符号精度
方程组求解	Solver (SymPy)	需代数精度
关系可满足性判定	Solver (Z3)	需严格逻辑
几何定理形式化应用	自研引擎	需领域规则
反例搜索	Solver (Z3)	需搜索
证明步骤自然语言化	LLM	需语言生成
形式化校验	Lean	需机器证明

8.1 协作流程



9. 失败处理

失败	处理
SymPy 无解	返回 false + reason, LLM reflect
Z3 unknown	标记”无法判定”，降级数值验证
引擎无匹配规则	LLM 显式引导
Lean 校验失败	定位错误步骤，LLM 修正
数值与符号不一致	报警，优先信符号（可能感知误差）

10. 设计路线小结

Symbolic Solver 的设计路线为：“**定理库 (RAG) + SymPy(代数) + Z3(约束) + 自研引擎 (规则前向链锁) + Lean(形式化)**”多组件协同。LLM 负责规划与语言化，Solver 负责精确计算与规则推导，二者形成”规划—计算—验证”的可靠分工，保证证明的数学严格性。